DS	**Session**	Graph	**Question Information**	Level 1 Challenge 82

Question description

Udaya family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input

The first input line has two integers n and m: the number of family members and toppings. The toppings are numbered 1,2,…,m.

After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x" (topping x is good) or "- x" (topping x is bad).

Output

Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

Constraints

- $1 \leq n,m \leq 10^5$
- $1 \leq x \leq m$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
```

EXPECTED OUTPUT

```
- + + + -
```

Test Case 2

INPUT (STDIN)

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
```

EXPECTED OUTPUT

```
- + - - +
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
600
```

Test Case 3

NLOC

```
71
```

Code

giant_pizza_2sat.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXV 200005

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head[MAXV];
Edge* rhead[MAXV];
Edge pool[8 * MAXV];
int pool_ptr = 0;

void link(int i, int j) {
    pool[pool_ptr].to = j;
    pool[pool_ptr].next = head[i];
    head[i] = &pool[pool_ptr++];

    pool[pool_ptr].to = i;
    pool[pool_ptr].next = rhead[j];
    rhead[j] = &pool[pool_ptr++];
}

int order[MAXV], order_sz = 0;
bool visited[MAXV];
int comp[MAXV], comp_cnt = 0;

void dfs1(int u) {
    visited[u] = true;
    for (Edge* e = head[u]; e; e = e->next) {
        if (!visited[e->to]) dfs1(e->to);
    }
    order[order_sz++] = u;
}

void dfs2(int u, int c) {
    comp[u] = c;
    for (Edge* e = rhead[u]; e; e = e->next) {
        if (comp[e->to] == 0) dfs2(e->to, c);
    }
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 0; i < n; i++) {
        char s1[5], s2[5];
        int u, v;
        scanf("%s %d %s %d", s1, &u, s2, &v);
        int lit1 = (s1[0] == '+') ? (2 * u) : (2 * u + 1);
        int lit2 = (s2[0] == '+') ? (2 * v) : (2 * v + 1);
        int not_lit1 = lit1 ^ 1;
        int not_lit2 = lit2 ^ 1;

        link(not_lit1, lit2);
        link(not_lit2, lit1);
    }

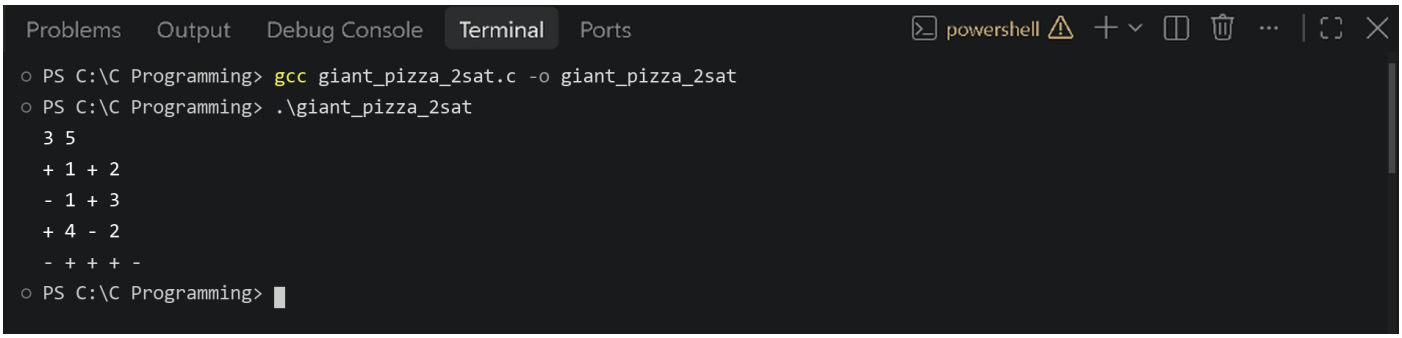
    int total_nodes = 2 * m + 2;
    for (int i = 2; i < total_nodes; i++) {
        if (!visited[i]) dfs1(i);
    }
    for (int i = order_sz - 1; i >= 0; i--) {
        int u = order[i];
        if (comp[u] == 0) dfs2(u, ++comp_cnt);
    }

    for (int i = 1; i <= m; i++) {
        if (comp[2 * i] == comp[2 * i + 1]) {
            printf("IMPOSSIBLE\n");
            return 0;
        }
    }

    for (int i = 1; i <= m; i++) {

        if (comp[2 * i] > comp[2 * i + 1]) {
            printf("+ ");
        } else {
            printf("- ");
        }
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc giant_pizza_2sat.c -o giant_pizza_2sat
PS C:\C Programming> .\giant_pizza_2sat
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
- + + + -
PS C:\C Programming>